



INGENIERÍA DE SISTEMAS –SISTEMAS OPERATIVOS-UFPS INTRODUCCIÓN A LA ADMINISTRACIÓN DE ARCHIVOS

Objetivo: * Conocer los principales utilitarios para la manipulación de ficheros en el S.O Linux.

Introducción:

Bajo la mayoría de los sistemas operativos (UNIX,LINUX incluido), existe el concepto de fichero, el cual es un conjunto de información al que se le ha asignado un nombre (llamado nombre del fichero). Ejemplos de fichero son un mensaje de correo, o un programa que puede ser ejecutado. Esencialmente, cualquier cosa salvada en el disco es guardada en un fichero individual.

El árbol de directorios:

La mayoría de los sistemas LINUX tienen una distribución de ficheros estándar, de forma que recursos y ficheros puedan ser fácilmente localizados. Esta distribución forma el árbol de directorios, el cual comienza en el directorio "/", también conocido como "directorio raíz". Directamente por debajo de / hay algunos subdirectorios importantes: /bin, /etc, /dev y /usr, entre otros. Estos a su vez contienen otros directorios con ficheros de configuración del sistema, programas, etc. En particular, cada usuario tiene un directorio "home". Este es el directorio en el que el usuario guardara sus ficheros. Por ejemplo (/home/grupoa01/).

1. UTILITARIOS DE MANIPULACIÓN DE ARCHIVOS

En este contexto, la manipulación de archivos significa copiar, mover y borrar archivos. Más adelante, veremos formas de cambiar los atributos de los mismos (dueño, permisos asociados).

1.1. mkdir, touch (tocar): creación de directorios y archivos vacíos. mkdir (MaKe DIRectory, Crear directorio) se usa para crear directorios. Su sintaxis es simple:

mkdir [opciones] <directorio> [directorio ...]

Sólo una opción es digna de interés: la opción -p. La misma hace dos cosas:

- creará los directorios padre si es que aún no existían. Si no se especifica esta opción y los directorios padre no existen, mkdir simplemente fallará, quejándose que dichos directorios padre no existen;
- retornará silenciosamente si el directorio que desea crear ya existe. Similarmente, si no especificó la opción -p, mkdir retornará un mensaje de error, quejándose que el directorio ya existe.

Por ejemplo:

mkdir pepe crea un directorio denominado pepe en el directorio corriente;

mkdir -p images/misc docs crea un directorio misc en el directorio images creando primero el último si es que no existe (-p); también crea un directorio denominado docs en el directorio corriente.

• (CREANDO FICHEROS) TOUCH:

Inicialmente, el comando touch no está orientado a la creación de archivos sino a la actualización de la hora de acceso y modificación de los archivos[1]. Sin embargo, touch creará los archivos mencionados como archivos vacíos si es que no existían. La sintaxis es:



INGENIERÍA DE SISTEMAS –SISTEMAS OPERATIVOS-UFPS INTRODUCCIÓN A LA ADMINISTRACIÓN DE ARCHIVOS

touch [opciones] archivo [archivo...]

Entonces, ejecutar el comando:

`touch archivo1 imagenes/archivo2`, creará un archivo vacío denominado `archivo1` en el directorio corriente y un archivo vacío denominado `archivo2` en el directorio imágenes.

1.2. rm : borrar archivos o directorios. El comando `rm` (ReMove, Quitar) reemplaza a los comandos del `y deltree` de DOS, y agrega más opciones. Su sintaxis es la siguiente:

`rm [opciones] <archivo|directorio> [archivo|directorio...]`

Las opciones incluyen:

`-r`, o `-R`: borrar recursivamente. Esta opción es obligatoria para borrar un directorio, vacío o no. Sin embargo, también puede usar el comando `rmdir` para borrar directorios vacíos.

`-i`: pedir confirmación antes de cada supresión. Note que predeterminadamente en Mandrake Linux, `rm` es un alias a `rm -i`, por razones de seguridad (existen alias similares para los comandos `cp` y `mv`). Estos alias le pueden ser más o menos útiles de acuerdo a su experiencia. Si desea quitarlos, puede editar su `.bashrc` y agregar esta línea: `unalias rm cp mv`.

`-f`: la opuesta de `-i`, fuerza la supresión de los archivos o directorios, incluso si el usuario no tiene derecho de escritura sobre los archivos[2].

Algunos ejemplos:

`rm -i imagenes/*.jpg` `archivo1`: borra todos los archivos cuyo nombre termina en `.jpg` en el directorio `imagenes` y borra el archivo `archivo1` en el directorio corriente, pidiendo confirmación para cada uno de los archivos. Responda `y` para confirmar la supresión, `n` para cancelarla.

`rm -Rf images/misc/` `archivo*`: borra todo el directorio `misc/` del directorio `images/` junto con todos los archivos del directorio corriente cuyos nombres comienzan con `archivo` sin pedir confirmación alguna.

1.3. mv : mover o renombrar archivos. La sintaxis del comando `mv` (MoVe, mover) es la siguiente:

mv [opciones] <archivo|dir.> [archivo|dir. ...] <destino>

Algunas opciones:

`-f`: fuerza la operación – no hay advertencia alguna en caso de que la operación sobre-escriba un archivo que ya existe.



INGENIERÍA DE SISTEMAS –SISTEMAS OPERATIVOS-UFPS INTRODUCCIÓN A LA ADMINISTRACIÓN DE ARCHIVOS

- i: lo contrario – pedir confirmación al usuario antes de sobre-escribir un archivo existente.
- v: modo verboso, reportar todos los cambios y la actividad.

Algunos ejemplos:

`mv -i /tmp/pics/*.png .:` mover todos los archivos del directorio `/tmp/pics/` cuyos nombres terminan en `.png` al directorio corriente (`.`), pidiendo confirmación antes de sobre-escribir cualquier archivo.

`mv pepe pupu:` cambiar el nombre del archivo `pepe` por `pupu`. Si ya hubiera un directorio `pupu`, el efecto de este comando sería mover todo el directorio `pepe` (el directorio en sí mismo más todos los archivos y directorios que contenga, recursivamente) dentro del directorio `pupu`.

`mv -vf archivo* imágenes/ tacho/:` mover, sin pedir confirmación, todos los archivos del directorio corriente cuyos nombres comienzan con `archivo` junto con todo el directorio `imágenes/` al directorio `tacho/`, y mostrar cada operación llevada a cabo.

1.4. cp : copiar archivos y directorios. `cp` (CoPy, Copiar) reemplaza a los comandos `copy`, `xcopy` de DOS, y agrega más opciones. Su sintaxis es la siguiente:

`cp [opciones] <archivo|dir.> [archivo|dir. ...] <destino>`

Algunas opciones:

- R: copiar recursivamente; obligatoria para copiar un directorio, incluso si está vacío.
- i: pedir confirmación antes de sobre-escribir cualquier archivo que pudiera sobre-escribirse.
- f: lo opuesto de `-i`, reemplazar cualquier archivo existente sin pedir confirmación alguna.
- v: modo "verboso", reporta todas las acciones que realiza `cp`.

Algunos ejemplos:

`cp -i /tmp/imagenes/* imagenes/:` copia todos los archivos del directorio `/tmp/imagenes` al directorio `imagenes/` ubicado en el directorio corriente. Si se va a sobre-escribir un archivo se pide confirmación.

`cp -vR docs/ /shared/mp3s/* miscosas/:` copia todo el directorio `docs` al directorio actual más todos los archivos del directorio `/shared/mp3s` al directorio `miscosas` ubicado en el directorio corriente.

`cp pepe pupu:` hace una copia del archivo `pepe` bajo el nombre `pupu` en el directorio corriente.

Notas:

- Hay tres etiquetas de tiempo distintas para cada archivo en LINUX: la última fecha de acceso al mismo (`atime`), es decir, la fecha cuando se abrió para lectura o escritura; la última fecha cuando se modificaron los atributos del i-nodo (`mtime`); y finalmente, la última fecha cuando se modificó el contenido del archivo (`ctime`).



INGENIERÍA DE SISTEMAS –SISTEMAS OPERATIVOS-UFPS INTRODUCCIÓN A LA ADMINISTRACIÓN DE ARCHIVOS

- Es suficiente que un usuario no privilegiado tenga derecho de escritura sobre un directorio para que pueda borrar los archivos que se encuentran en el mismo, incluso si dicho usuario no es el dueño de los archivos.

1.5 cd <directorio>, donde <directorio> es el nombre del directorio al que queremos ir.

Si el nombre de un fichero termina con un carácter distinto a "/", el sistema supone que se está refiriendo al fichero con su posición relativa a su directorio de trabajo. Esto es conocido como camino relativo. Por otra parte, si comienza el nombre del fichero con "/", el sistema interpreta esto como un camino completo es decir, el camino al fichero completo desde el directorio raíz, /. Esto es conocido como camino absoluto.

Cada directorio tiene una entrada de nombre "." la cual se refiere al directorio padre. De igual forma, existe en cada directorio la entrada "." la cual se refiere a sí mismo. Así que el comando nos deja donde estamos. También, usando cd sin argumentos nos llevara a nuestro directorio de origen.

1.6 Ls :

Muestra información sobre los ficheros o directorios indicados. Sintaxis: ls <fichero1> <fichero2> ...<ficheroN> Donde <fichero1> a <ficheroN> son los ficheros o directorios a listar.

2. PATRONES DE ENGLOBAMIENTO DEL SHELL

Una característica importante de la mayoría de los intérpretes de comandos en Linux es la capacidad para referirse a más de un fichero usando caracteres especiales. Estos llamados comodines le permiten referirse a, por ejemplo, todos los ficheros que contienen el carácter "n". El comodín "*" hace referencia a cualquier carácter o cadena de caracteres en el fichero. Por ejemplo, cuando usa el carácter "*" En el nombre de un fichero, el intérprete de comandos lo sustituye por todas las combinaciones posibles provenientes de los ficheros en el directorio al cual nos estamos refiriendo.

El proceso de la sustitución de "*" en nombres de ficheros es llamado expansión de comodines y es efectuado por el intérprete de comandos. Esto es importante: las órdenes individuales, como ls, nunca ven el "*" en su lista de parámetros. Es el intérprete quien expande los comodines para incluir todos los nombres de ficheros que se adaptan. Luego la orden Carácter comodín es "?". Este carácter comodín solo expande un único carácter. Luego "ls ?" mostrará todos los nombres de ficheros con un carácter de longitud, y "ls termca?" mostrara "termcap" pero no "termcap.backup".

Cuando se ingresa un comando como ls *.txt y presiona Intro, la tarea de encontrar cuales archivos se corresponden con el patrón *.txt no la realiza el comando ls, sino el shell en sí mismo. Esto requiere de una pequeña explicación sobre cómo interpreta el shell la línea de comandos. Cuando Usted ingresa:

```
$ ls *.txt
leerme.txt recetas.txt
```



INGENIERÍA DE SISTEMAS –SISTEMAS OPERATIVOS-UFPS INTRODUCCIÓN A LA ADMINISTRACIÓN DE ARCHIVOS

Primero la línea de comandos se separa en palabras (ls y *.txt, en este ejemplo). Cuando el shell ve un * en una palabra, interpretará toda la palabra como un patrón de englobamiento y la reemplazará con los nombres de todos los archivos que se correspondan con el patrón. Por lo tanto, justo antes que el shell la ejecute, la línea se convirtió en la línea ls leerme.txt recetas.txt, lo que da el resultado esperado. El shell reacciona así frente a otros caracteres como:

? se corresponde con un único carácter (uno y sólo uno), cualquiera que sea este;

[...] se corresponde con cualquiera de los caracteres que se encuentran entre los corchetes; los caracteres pueden estar referidos por intervalos (por ejemplo, 1-9) o por valores discretos, o una mezcla de ambos. Ejemplo: [a-zBE5-7] se corresponderá con todos los caracteres desde la a hasta la z, una B, una E, un 5, un 6 o un 7;

[!...]: se corresponde con cualquier carácter que no se encuentre en los corchetes. [!a-z], por ejemplo, se corresponderá con cualquier carácter que no sea una letra minúscula[1];

{c1,c2} se corresponde con c1 o con c2, donde c1 y c2 también son patrones de englobamiento, lo cual significa que Usted puede escribir {[0-9]*,[acr]} por ejemplo.

Algunos ejemplos:

/etc/*conf: todos los archivos del directorio /etc cuyo nombre termine con conf. Se puede corresponder con /etc/inetd.conf, pero también con /etc/conf.linuxconf y también con /etc/conf si existe tal archivo: recuerde que * puede corresponderse con una cadena vacía.

imagen/{autos,espacio[0-9]}/*.jpg: todos los archivos cuyo nombre termina en .jpg en los directorios imagen/autos, imagen/espacio0, ... , imagen/espacio9, si es que dichos directorios existen.

/usr/share/doc/*/LEAME: todos los archivos denominados LEAME en todos los subdirectorios inmediatos del directorio /usr/share/doc. Esto hará que /usr/share/doc/mandrake/LEAME corresponda por ejemplo, pero no /usr/share/doc/miprogram/doc/LEAME.

*[!a-z] Todos los archivos en el directorio corriente cuyo nombre no termine con una letra minúscula.

3. EL COMPLETADO DE LA LÍNEA DE COMANDOS

El completado es una funcionalidad muy útil, y todos los shells modernos (bash, inclusive) la tienen. Su rol es darle al usuario el menor trabajo posible. La mejor manera de ilustrarlo es con un ejemplo.



INGENIERÍA DE SISTEMAS –SISTEMAS OPERATIVOS-UFPS

INTRODUCCIÓN A LA ADMINISTRACIÓN DE ARCHIVOS

3.5.1 Ejemplo. Suponga que su directorio personal contiene un archivo cuyo nombre es `archivo_con_un_nombre_muy_largo`, y usted quiere mirarlo. Suponga que Usted también tiene en el mismo directorio otro archivo denominado `archivo_texto`. Usted está en su directorio personal. Así que usted ingresa la secuencia siguiente:

```
$ less ar<TAB>
```

(es decir, ingresa `less ar` y luego presiona la tecla `TAB`). El shell ahora extenderá la línea de comandos por Usted:

```
$ less archivo_
```

y también le dará la lista de elecciones posibles (en su configuración predeterminada, que se puede personalizar). Luego ingrese la siguiente secuencia de teclas:

`less archivo_c<TAB>`, y el shell extenderá la línea de comandos para darle el resultado que Usted quiere:

```
less archivo_con_un_nombre_muy_largo
```

Entonces, todo lo que necesita hacer es presionar la tecla Intro para confirmar y leer el archivo.

3.5.2. Otros métodos de completado. La tecla `TAB` no es la única manera de activar el completado, aunque es la más común. Como regla general, la palabra a completar será el nombre de un comando para la primera palabra de la línea de comandos (`ns!<TAB>` dará `nslookup`), y el nombre de un archivo para todos los demás parámetros, a menos que la palabra esté precedida por un carácter "mágico" como `~`, `@` o `$`, en cuyo caso el shell intentará completar, respectivamente, un nombre de usuario, una máquina o una variable de entorno[*]. También hay un carácter mágico para completar el nombre de un comando (!) o el nombre de un archivo (/).

Las otras dos formas de activar el completado son las secuencias `Esc-<x>` y `Ctrl+x <x>`, donde `<x>` es uno de los caracteres mágicos ya mencionados. `Esc-<x>` intentará el completado de manera única; si falla completará la palabra con la sub-cadena más larga posible de la lista de opciones. Un bip significa que la opción no es única o simplemente que no hay opción correspondiente. La secuencia `Ctrl+x <x>` muestra la lista de opciones posibles sin intentar completar alguno. Presionar la tecla `TAB` es lo mismo que presionar sucesivamente `Esc-<x>` y `Ctrl+x <x>`, donde el carácter mágico depende del contexto.

Por lo tanto, una forma de ver todas las variables de entorno definidas es teclear la secuencia `Ctrl+x $` en una línea en blanco. Otro ejemplo: si desea ver la página `man` del comando `nslookup`, simplemente teclea `man ns!` luego `Esc-!`, y el shell completará automáticamente como `man nslookup`.



INGENIERÍA DE SISTEMAS –SISTEMAS OPERATIVOS-UFPS INTRODUCCIÓN A LA ADMINISTRACIÓN DE ARCHIVOS

Notas:

*Recuerde: LINUX existen diferencias entre mayúsculas y minúsculas.